Détection d'anomalies dans un réseau d'aéroports avec des GNN

Nour Ben Mansour, Mohamed ali Damergi and Léo Morin

Université Claude Bernard Lyon 1, France

Abstract. We look for erroneous links and nodes in an airport network (3363 nodes, 13547 edges). For links, we inject random false edges and rank edges by the reconstruction probability of a GAE or VGAE: AUC is 0.92–0.94, but Adamic–Adar, which needs no training, does better in AUC. For nodes, a GCN-based autoencoder (DOMINANT) and a GAT predicting population reach an AUC of about 0.90 on injected anomalies. On the original graph, the models find real data errors, such as a node with wrong coordinates and default populations. Code: `https://github.com/LeoMor1/projet-GNN_2026-2027`.

Keywords: Graph neural networks, graph autoencoder, link anomaly detection

1 Introduction

Le graphe étudié relie 3363 aéroports par 13547 liaisons non orientées ; chaque nœud a une latitude, une longitude, la population de sa ville, un nom de ville et un pays. Une partie de ces informations est fausse, et certains liens aussi. Nous comparons ce qu'un modèle prédit à ce qui est observé et regardons où les deux divergent le plus : liens (GAE, VGAE), puis nœuds (GCN, GAT). Un autoencodeur de graphe repère-t-il les liens aberrants mieux qu'une heuristique sans apprentissage ? Un GCN (DOMINANT) repère-t-il les attributs mal reconstruits, et un GAT les populations qui s'écartent de sa prédiction ? Code : `https://github.com/LeoMor1/projet-GNN_2026-2027`.

2 État de l'art

Liens. Pour juger un lien, on peut comparer les voisinages de ses extrémités : voisins communs, Jaccard, ou Adamic–Adar [1], qui pondère chaque voisin commun par $1/\log(\deg)$. Ces références de la prédiction de liens [6] n'utilisent ni attributs ni apprentissage. Le GAE et le VGAE [4] associent un encodeur GCN [5] à un décodeur $p_{ij} = \sigma(z_i^\top z_j)$; le VGAE rend z_i aléatoire et ajoute un terme KL. Pour les graphes attribués, DOMINANT [2] reconstruit structure et attributs avec un autoencodeur GCN (revue : [7]). Baselines : Jaccard, Adamic–Adar, distance entre aéroports.

Nœuds. Un nœud est suspect quand ses attributs ou ses voisins s'écartent de ce que le reste du graphe laisse attendre [7]. DOMINANT [2] score chaque nœud par son erreur de reconstruction (attributs et structure) avec un encodeur GCN [5]. Le GAT [8] pondère les voisins par des coefficients d'attention appris. GraphRfi [9] repère les fraudeurs par leur erreur de prédiction, principe que nous reprenons pour la population. Baselines : moyenne globale et moyenne des voisins.

3 Anomalies de liens

3.1 Protocole et modèle

Faute d'étiquette « lien faux », nous ajoutons $r\,|E|$ liens entre paires non reliées tirées au hasard, $r \in \{1, 5, 10, 20\}\,\%$, *avant* le découpage (`RandomLinkSplit`, 10 % validation, 20 % test). Entraînement par reconstruction sur 70 % des liens (injectés compris) ; score $1-\sigma(z_i^\top z_j)$. Entrées : lat, lon, population, standardisées. Le GAE a deux couches GCN ($3 \to 64 \to 32$) ; le VGAE ajoute deux couches parallèles (μ, $\log\sigma$) et un terme KL. Adam, PyTorch Geometric [3], 5 graines ; métriques AUC et AP.

3.2 Résultats

Table 1. Liens injectés : moyenne sur 5 graines, écart-type entre parenthèses ($0{,}952(4) = 0{,}952 \pm 0{,}004$). Gras : meilleur. AP aléatoire : 0,010 à 0,167 ($r/(1+r)$).

	$r = 1\,\%$		$r = 5\,\%$		$r = 10\,\%$		$r = 20\,\%$	
Méthode	AUC	AP	AUC	AP	AUC	AP	AUC	AP
Distance (km)	0,914(12)	0,080(7)	0,916(5)	0,298(9)	0,915(3)	0,452(6)	0,916(3)	0,613(7)
Jaccard	0,949(6)	0,098(3)	0,947(2)	0,349(3)	0,946(2)	0,516(4)	0,946(1)	0,679(2)
Adamic–Adar	**0,952**(4)	0,098(3)	**0,951**(2)	0,351(3)	**0,951**(1)	0,519(3)	**0,950**(1)	**0,683**(2)
GAE	0,945(7)	0,190(25)	0,936(4)	0,429(19)	0,929(4)	**0,552**(8)	0,917(3)	0,650(7)
VGAE	0,943(9)	**0,193**(14)	0,933(3)	**0,434**(15)	0,927(4)	0,551(10)	0,916(2)	0,654(5)

Adamic–Adar a le meilleur AUC à tous les taux (0,95 contre 0,917 à 0,945) sans apprentissage (tableau 1). Les autoencodeurs ne gardent l'avantage en AP que si les anomalies sont rares. GAE et VGAE diffèrent de moins de 0,005, comme l'écart-type : le VGAE n'apporte rien de démontré.

Sur le graphe d'origine (graine 42), les dix premiers liens relient surtout un grand aéroport à une destination éloignée (Paris–Cayenne), probablement réels. Une vraie erreur : le VGAE classe cinquième Ottawa–Iqaluit, or le nœud « Ottawa » est placé en (−27,9; 144,6), en Australie, alors que ses voisins sont nord-américains : le lien est correct, les attributs du nœud sont faux.

3.3 Évaluation et ablation

Une couche ou l'absence d'attributs dégradent le GAE (tableau 2) ; sans standardisation, la population ($\sim 10^7$) sature probablement le sigmoïde.

Table 2. Ablation du GAE ($r = 10\,\%$, 5 graines).

	Complet	1 couche GCN	Sans standardisation	Sans attributs	$\dim Z = 8$
AUC	0,929(4)	0,846(10)	0,500(0)	0,812(3)	0,930(3)
AP	0,552(8)	0,365(14)	0,091(0)	0,202(4)	0,558(16)

4 Anomalies de nœuds : GCN

4.1 Protocole et modèle

Sans étiquette « nœud faux », nous entraînons DOMINANT [2] sur le graphe complet : un nœud est d'autant plus suspect qu'il est mal reconstruit. Les entrées sont $F = 7$ attributs standardisés : position sur la sphère unité (x, y, z), $\log(1 + \text{pop})$ avec indicateur de population manquante (défaut 10 000), et $\log(1+d)$ pour les distances moyenne et minimale aux voisins. L'encodeur a trois couches GCN ($7 \to 64 \to 32 \to 16$), le décodeur d'attributs deux couches ($16 \to 32 \to 7$), le décodeur structurel une couche puis $\sigma(\tilde{Z}\tilde{Z}^\top)$. Le score d'un nœud somme ses deux erreurs de reconstruction, recentré médiane/MAD. Adam (0,005, 500 époques), 5 entraînements dont les scores sont moyennés.

4.2 Résultats

L'erreur d'attributs passe de 0,9997 à 0,38, la structure de 1,10 à 0,80 ; les cinq entraînements convergent à moins de 0,01. Neuf des dix nœuds les plus mal reconstruits sont mal géolocalisés (Hasvik en Norvège placé en Nouvelle-Zélande, St Thomas aux Caraïbes en Australie, Wewak en Papouasie au Canada) : les features les plus mal reconstruites sont positionnelles (r_{if} de 27 à 55). Frankfurt fait exception : bien placé, mais avec une population par défaut 10 000 sous-dimensionnée — une erreur de donnée, pas de position. Les liens ne sont pas en cause : pour ces dix nœuds, le lien existant le moins plausible garde une probabilité $\geq 0{,}71$, et les liens absents les plus attendus ($\text{p} > 0{,}99$) vont vers les hubs de leur région réelle : l'embedding retrouve la région malgré des coordonnées fausses. Ottawa relie cette analyse à la section 3 : le VGAE y classait haut le lien Ottawa–Iqaluit alors que ce sont les attributs du nœud qui sont faux.

4.3 Évaluation et ablation

Par injection de $r \in \{1, 5, 10, 20\}\,\%$ de nœuds corrompus, le modèle atteint une AUC de 0,95 ($r = 1\,\%$) à 0,87 ($r = 20\,\%$) et une AP de 0,20 à 0,61 (moyenne sur 5 entraînements). La performance décroît avec r car les nœuds corrompus participent à l'entraînement, l'erreur d'attributs passant de 0,39 à 0,50 entre $r = 0$ et $20\,\%$. Sans prétraitement (lat/lon/pop brutes), la méthode tombe au niveau du hasard (AUC $\approx 0{,}49$, AP $\approx r$), la perte étant dominée par la population brute : la détection repose sur la conception des attributs, pas sur l'architecture seule.

5 Anomalies de nœuds : GAT

5.1 Protocole et modèle

Comme GraphRfi [9], qui repère les fraudeurs par leur erreur de prédiction, nous *prédisons* la population d'un aéroport à partir de sa position et de ses liaisons : un nœud est suspect quand la population observée s'écarte de la prédiction. La population est donc la cible $y_i = \log(1+\text{pop}_i)$, standardisée, et jamais une entrée. Les entrées sont lat, lon et $\log(1+\deg)$, standardisées. Une couche GAT [8] fait une moyenne des voisins pondérée par des coefficients d'attention appris, avec K têtes concaténées :

$$\alpha_{ij} = \text{softmax}_j\left(\text{LeakyReLU}(a^\top [Wh_i \,\|\, Wh_j])\right), \qquad h'_i = \Big\|_{k=1}^{K} \sum_j \alpha_{ij}^k W^k h_j.$$

Comme $\sum_j \alpha_{ij} = 1$, le GAT ne voit pas le nombre de voisins, d'où le degré en entrée. Deux couches ($3 \to 64$ avec $K = 4$, puis $256 \to 64$ avec $K = 1$), ELU, dropout 0,2 et une couche linéaire donnent $\hat{y}_i$. Perte MSE sur 80 % des nœuds, Adam (0,005, 200 époques), graines de la section 3. Le score est $s_i = |\hat{y}_i - y_i|$.

Pour évaluer, nous multiplions ou divisons la population de rN nœuds par un facteur entre 10 et 100 (ils restent dans l'entraînement), puis calculons AUC et AP sur les 20 % de nœuds test. Les baselines remplacent $\hat{y}_i$ par la moyenne des nœuds d'entraînement ou des voisins.

5.2 Résultats

Table 3. Populations injectées, nœuds test, moyenne sur 5 graines (écart-type sur le dernier chiffre entre parenthèses). Une AP aléatoire vaut environ r.

	$r = 1\,\%$		$r = 5\,\%$		$r = 10\,\%$		$r = 20\,\%$	
Méthode	AUC	AP	AUC	AP	AUC	AP	AUC	AP
Moyenne globale	0,881(77)	0,297(133)	0,828(45)	0,373(77)	0,807(43)	0,499(89)	0,823(19)	0,665(20)
Moyenne des voisins	0,792(77)	0,169(48)	0,791(61)	0,253(100)	0,777(34)	0,343(64)	0,762(14)	0,460(45)
GAT	**0,911**(32)	**0,524**(115)	**0,896**(22)	**0,575**(80)	**0,900**(26)	**0,698**(75)	**0,899**(16)	**0,790**(17)

Le GAT bat les deux baselines à tous les taux (tableau 3), avec une AUC stable vers 0,90 : ne voyant pas la population en entrée, il ne peut pas apprendre à recopier les valeurs faussées. En prédiction, le GAT atteint $R^2 = 0{,}37(3)$ sur les nœuds test, et l'ablation le fait baisser à chaque fois : 0,29 sans le degré, 0,32 sans les liens (degré gardé), 0,31 avec une couche et 0,34 avec une tête. Le degré compte le plus, mais les voisins aident aussi.

Sur le graphe d'origine (graine 42), le top 20 mêle des mégapoles sous-estimées (Delhi, Karachi) et des erreurs de données : Francfort (234 liaisons), Calcutta et deux autres villes à exactement 10 000 habitants, valeur par défaut de 1673 nœuds sur 3363, et Hamilton (Nouvelle-Zélande) avec 902 habitants, probablement ceux de Hamilton aux Bermudes.

Table 4. Les trois approches (injection de $r = 1$ à 20 % d'anomalies ; AUC et AP du $r = 1\,\%$ au $r = 20\,\%$).

Modèle	Anomalie ciblée	Score	AUC	AP
GAE	liens ajoutés	$1 - p_{ij}$	0,945 → 0,917	0,19 → 0,65
GCN (DOMINANT)	attributs, position	reconstruction	0,95 → 0,87	0,20 → 0,61
GAT	population	$\lvert\hat{y}_i - y_i\rvert$	0,911 → 0,899	0,52 → 0,79

6 Comparaison et conclusion

Les anomalies visées diffèrent, donc les AUC ne sont pas directement comparables, mais deux constats se recoupent. La qualité des entrées décide de tout : sans standardisation ou prétraitement, le GAE et DOMINANT tombent au niveau du hasard (AUC 0,5 et 0,49). Et sur le graphe réel, les modèles retrouvent de vraies erreurs : Ottawa placé en Australie, Francfort à 10 000 habitants (valeur par défaut de 1673 nœuds), Hamilton (Nouvelle-Zélande) avec la population de Hamilton aux Bermudes.

Limites : anomalies injectées aléatoires donc faciles (Adamic–Adar bat les autoencodeurs en AUC), tops issus d'une seule graine. À faire : vérifier les tops sur OpenFlights, combiner les trois scores.

Références

1. Adamic, L.A., Adar, E. : Friends and neighbors on the Web. Social Networks 25(3), 211–230 (2003)
2. Ding, K., Li, J., Bhanushali, R., Liu, H. : Deep anomaly detection on attributed networks. In : Proceedings of the SIAM International Conference on Data Mining (SDM). pp. 594–602 (2019)
3. Fey, M., Lenssen, J.E. : Fast graph representation learning with PyTorch Geometric. In : ICLR Workshop on Representation Learning on Graphs and Manifolds (2019)
4. Kipf, T.N., Welling, M. : Variational graph auto-encoders. In : NIPS Workshop on Bayesian Deep Learning (2016)
5. Kipf, T.N., Welling, M. : Semi-supervised classification with graph convolutional networks. In : International Conference on Learning Representations (ICLR) (2017)
6. Liben-Nowell, D., Kleinberg, J. : The link-prediction problem for social networks. Journal of the American Society for Information Science and Technology 58(7), 1019–1031 (2007)
7. Ma, X., Wu, J., Xue, S., Yang, J., Zhou, C., Sheng, Q.Z., Xiong, H., Akoglu, L. : A comprehensive survey on graph anomaly detection with deep learning. IEEE Transactions on Knowledge and Data Engineering 35(12), 12012–12038 (2023)
8. Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y. : Graph attention networks. In : International Conference on Learning Representations (ICLR) (2018)
9. Zhang, S., Yin, H., Chen, T., Hung, Q.V.N., Huang, Z., Cui, L. : GCN-based user representation learning for unifying robust recommendation and fraudster detection. In : Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval. pp. 689–698 (2020)